Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan
College of Engineering
(An Autonomous Institution affiliated to
Rashtrasant Tukadoji Maharaj Nagpur
University)

Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Nagar Yuwak Shikshan Sanstha's

Yeshwantrao Chavan College of Engineering

(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)

Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2024-25 (ODD)**	**Course Name**	**Computer vision Lab**
Semester	4	**Course Code**	AIDS
Roll No	120	**Name of Student**	Shrusti Katakwar

Practical Number	01
Course Outcome	1. Understand the fundamental concepts of non -linear data structures 2. Implement the real-life application using nonlinear data structures. 3. Apply the concept of hashing, disjoint set and union–find techniques with path compression to manage equivalence relations and dynamic groups 4. Analyze graph-based problems using traversal, topological sorting, and shortest path algorithms. 5. Evaluate the problem requirements and implement an optimal solution using relevant data structures.
Aim	Program based on tree data structure
Problem Definition	A Library Management System (LMS) manages thousands of books. Each book is identified by a unique Book ID and associated metadata like title, author, etc. The system requires a fast and dynamic method to locate books, add new entries, and remove outdated ones. Design a database indexing system using a Binary Search Tree (BST) to: i. Store and maintain the Book IDs in sorted order.

	ii. Efficiently search for books using their Book IDs. iii. Support dynamic operations like adding new books or deleting books when they are no longer available. iv. Enable retrieval of books within a specific range of IDs for generating reports.
Theory (100 words)	A Library Management System manages thousands of books, each identified by a **unique Book ID**. To efficiently store, search, insert, delete, and retrieve books within a range, a **Binary Search Tree (BST)** can be used as a database indexing structure. A BST is a hierarchical data structure that organizes data in a sorted manner, allowing efficient dynamic operations.
	2. Need for BST in LMS The system requires: • Fast searching of books by Book ID • Maintenance of Book IDs in sorted order • Dynamic insertion of new books • Deletion of unavailable books • Efficient range-based retrieval for reports A BST satisfies all these requirements effectively. **3. Structure of the BST Index** Each node of the BST represents one book record and contains: • Book ID (Primary Key) • Title • Author • Publication details • Reference to left child • Reference to right child The **Book ID acts as the key** for organizing the tree. **4. Properties of Binary Search Tree** The BST follows these ordering rules:

	1. All Book IDs in the left subtree are less than the root's Book ID. 2. All Book IDs in the right subtree are greater than the root's Book ID. 3. No duplicate Book IDs are allowed. 4. Both left and right subtrees are also BSTs. Because of these properties, the tree always maintains sorted order.
Procedure and Execution (100 Words)	Algorithm: Start. Create a structure Node containing: • Integer bookID • Pointer left • Pointer right Initialize root = NULL. To insert a book: • If root is NULL, create a new node and assign it to root. • Otherwise, compare the new Book ID with current node: o If smaller, move to left subtree. o If greater, move to right subtree. o Repeat until correct position is found. • Insert the new node at that position. To search for a book: • Start from root. • If root is NULL, book not found. • If Book ID equals current node, return found. • If smaller, move to left subtree. • If greater, move to right subtree. • Repeat until found or NULL reached. To delete a book: • Search for the node with given Book ID. • If not found, stop. • If node has no child, delete it. • If node has one child, replace node with its child. • If node has two children: o Find minimum node in right subtree. o Replace current node value with that minimum value. o Delete the duplicate node from right subtree. To display books in sorted order: • Traverse left subtree. • Visit root node.

- Traverse right subtree.

 To retrieve books within a range:

- If current node value is greater than low, search left subtree.
- If current node value is within range, print it.
- If current node value is less than high, search right subtree.

 Stop.

Code:

```
#include <iostream>
using namespace std;

// Node structure
struct Node {
   int bookID;
   Node* left;
   Node* right;

   Node(int id) {
      bookID = id;
      left = right = NULL;
   }
};

// Insert a book
Node* insert(Node* root, int id) {
   if (root == NULL)
      return new Node(id);
   if (id < root->bookID)
      root->left = insert(root->left, id);
   else if (id > root->bookID)
      root->right = insert(root->right, id);
   return root;
}

// Search a book
bool search(Node* root, int id) {
   if (root == NULL) return false;
   if (root->bookID == id) return true;
   if (id < root->bookID) return search(root->left, id);
   return search(root->right, id);
}

// Find minimum node
Node* findMin(Node* root) {
   while (root->left != NULL)
      root = root->left;
```

```
    return root;
}

// Delete a book
Node* deleteNode(Node* root, int id) {
    if (root == NULL) return root;
    if (id < root->bookID)
        root->left = deleteNode(root->left, id);
    else if (id > root->bookID)
        root->right = deleteNode(root->right, id);
    else {
        if (!root->left) {
            Node* temp = root->right;
            delete root;
            return temp;
        } else if (!root->right) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        Node* temp = findMin(root->right);
        root->bookID = temp->bookID;
        root->right = deleteNode(root->right, temp->bookID);
    }
    return root;
}

// Inorder traversal
void inorder(Node* root) {
    if (root) {
        inorder(root->left);
        cout << root->bookID << " ";
        inorder(root->right);
    }
}

// Range search
void rangeSearch(Node* root, int low, int high) {
    if (!root) return;
    if (low < root->bookID)
        rangeSearch(root->left, low, high);
    if (root->bookID >= low && root->bookID <= high)
        cout << root->bookID << " ";
```

```
    if (high > root->bookID)
        rangeSearch(root->right, low, high);
}

// Main function
int main() {
    Node* root = NULL;

    // Insert books
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    cout << "Books in sorted order: ";
    inorder(root);

    cout << "\nSearch for Book ID 40: ";
    cout << (search(root, 40) ? "Found" : "Not Found");

    cout << "\nDelete Book ID 30\n";
    root = deleteNode(root, 30);

    cout << "Books after deletion: ";
    inorder(root);

    cout << "\nBooks between 40 and 75: ";
    rangeSearch(root, 40, 75);

    return 0;
}
```

Output:

```
Output

Books in sorted order: 20 30 40 50 60 70 80
Search for Book ID 40: Found
Delete Book ID 30
Books after deletion: 20 40 50 60 70 80
Books between 40 and 75: 40 50 60 70

=== Code Execution Successful ===
```

Output Analysis	The program first inserts the Book IDs 50, 30, 70, 20, 40, 60, and 80 into a Binary Search Tree. In a BST, smaller values go to the left and larger values go to the right. After inserting all values, the tree is properly arranged according to these rules. When the program performs an inorder traversal (Left → Root → Right), it prints the Book IDs in sorted order as: 20 30 40 50 60 70 80. This shows that the BST keeps the data sorted automatically. Next, the program searches for Book ID 40. It starts from the root and moves left or right depending on the value. Since 40 is found in the tree, the output shows "Found." Then the program deletes Book ID 30. Because 30 has two children, it is replaced with the next smallest value (40), and the duplicate node is removed. After deletion, the sorted output becomes: 20 40 50 60 70 80. Finally, the program prints the Book IDs between 40 and 75, which are: 40 50 60 70. Overall, the program successfully performs insertion, searching, deletion, and range display using a Binary Search Tree.
Link of student Github profile where lab assignment has	

Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

been uploaded	
Conclusion	The program successfully demonstrates how a Binary Search Tree (BST) can be used to manage Book IDs in a Library Management System. It stores the Book IDs in sorted order and allows efficient operations such as insertion, searching, and deletion. The inorder traversal automatically displays the books in ascending order, and the range search function helps retrieve books within a specific range of IDs. Overall, the BST provides a simple, fast, and dynamic way to organize and manage library records effectively.

Plag Report (Similarity index < 12%)	The attached file "plag-1.docx" appears to be a student lab report from YCCE Nagpu Department of Computer Technology, covering topics like BST for a Library Manag System, PEOs/POs, and including a pre-existing "Plag Report" section claiming a 12 similarity index. plag-1.docx I cannot generate or manipulate a custom plagiarism report to force a similarity inc under 12%, as that would violate academic integrity policies and my guidelines aga falsifying results. scribbr **Recommended Steps** • Use a free online plagiarism checker that supports DOCX uploads and has no/l limits, such as DupliChecker, PapersOwl, or SmallSEOTools. duplichecker +2 • Upload your file directly to one of these tools (e.g., https://www.duplichecker.c https://papersowl.com/free-plagiarism-checker) for an independent scan again and academic databases. papersowl +1 • Review matches, paraphrase if needed, and recheck to aim for under 12% simil check-plagiarism **File Observations** The document contains standard phrases from institutional templates (e.g., depart vision/mission, NAAC details) and lab descriptions that may flag as similar to publ sources like the college website. Common elements like PEO/PO keywords or BST t could contribute to any detected similarity. plag-1.docx
Date	19/02/26